

EMBEDDED SYSTEMS FUNDAMENTALS

From Bare Metal to Firmware

What we'll cover today:

- What a microcontroller really is
- Memory layout & the address space
- Startup: what runs before main()
- Peripherals & registers
- Cross-compilation & toolchains
- Debugging without printf

Credits: Claude

What is a Microcontroller?

What is a Microcontroller?

A computer vs a microcontroller

Your laptop / server

CPU

RAM (GBs)

Storage (SSD/HDD)

GPU

OS (Linux / Windows)

Drivers + stdlib

Each component is a separate chip on a motherboard

STM32 microcontroller (single chip)

ARM Cortex-M4 CPU

SPI / UART / I2C

Flash 512KB

Timers × 14

RAM 128KB

GPIO × 80

ADC / DAC

Everything is on one die

On an MCU, there is no OS to manage anything. Your firmware is the only program. You control every byte.

ARM Cortex-M

What 'Cortex-M' means and why it matters

ARM

ARM Holdings designs CPU cores but does not manufacture chips. They license the design.

Cortex-M

The 'M' profile = microcontroller. Optimised for low power, deterministic latency, no MMU.

M0 / M3 / M4 / M7

M0: cheapest, no hardware float. M4: hardware FPU. M7: dual-issue. HAVEN targets M0–M4.

STM32

STMicroelectronics licenses the Cortex-M core and wraps it with their own peripherals (SPI, UART etc.) to make the STM32 chip family.

Cortex-M register file (what the CPU actually has)

R0	General purpose / function arg 0
R1	General purpose / function arg 1
R2	General purpose / function arg 2
R3	General purpose / function arg 3
R4–R11	Caller-saved general purpose
R12	Intra-procedure scratch
SP (R13)	Stack pointer — points to top of stack
LR (R14)	Link register — holds return address
PC (R15)	Program counter — address of next instruction
xPSR	Status flags: N Z C V + exception state

No virtual memory, no protected mode, no privilege levels (in simple configs). One address space. One execution context.

Memory Layout

The Address Space

Everything the CPU can 'see' is a number from 0x00000000 to 0xFFFFFFFF

32-bit address space

0x00000000
Flash (code)
0x1FFF0000
System memory (ROM bootloader)
0x20000000
SRAM (your RAM)
0x40000000
Peripherals (SPI, UART, GPIO...)
0xE0000000
Cortex-M internals (NVIC, SysTick...)

Why does this matter?

The CPU uses the same 'read from address X' instruction whether it's reading your code, reading RAM, or reading a peripheral register. The memory controller routes the access to the right hardware based on the address range.

This is called memory-mapped I/O — and it means peripherals look like memory.

What lives where

Flash 0x0800_0000	Your compiled firmware binary (.text, .rodata, const arrays)
Flash — protected	K_master, node_id, key_rows — ARM RDP Level 2 seals this off
SRAM 0x2000_0000	neighbor_table[8], dedup_cache[16], election state, stack
0x4001_3000	SPI1 registers — how you talk to the nRF24L01+ radio chip
0xE000_E100	NVIC — interrupt controller for timer-driven election ticks

Flash vs SRAM — What Lives Where

FLASH (non-volatile)

Survives power-off. Read fast, write slow & limited (10k–100k cycles).

.text	Your compiled machine code. Read-only at runtime.
.rodata	String literals, const arrays, lookup tables.
secrets	K_master · node_id · key_rows Protected by ARM RDP Level 2 — physically unreadable post-deploy.
Interrupt vectors	Table of handler addresses at 0x00000000. CPU reads this on reset.
Startup code	Assembly that runs before main() — copies data, zeroes BSS.

SRAM (volatile)

Lost on power-off. Fast read/write. Every byte counts on Cortex-M.

.data	Initialised globals. Copied from flash to RAM at startup.
.bss	Uninitialised globals. Startup code zeroes this region.
HEAP	Dynamic allocation (malloc). AVOID — fragmentation kills embedded systems.
Stack	Grows downward. Function locals, return addresses. Fixed size — overflow = crash with no error message.
Runtime	neighbor_table[8] ~240B dedup_cache[16] ~100B election state ~32B All statically allocated.

Rule: if it needs to survive power-off, it's in flash or EEPROM. If it's runtime state, it's in SRAM. Never use malloc.

The volatile Keyword

One missing word → your code silently does the wrong thing

Without volatile — BROKEN

```
// Peripheral register address
uint32_t *SPI_SR = (uint32_t*)0x40013008;

// Wait for TX buffer empty
while (*SPI_SR & (1<<1)) { }

// Compiler sees: "this loop reads the same
// memory address every iteration and never
// writes to it — the condition can never
// change — I'll optimise it away"
// Result: infinite loop OR removed entirely
```

What the compiler sees:

```
// Compiler "optimises" to:
if (*SPI_SR & (1<<1)) {
    while(1) { } // or just: while(1);
}
// Your program hangs. No error. No warning.
```

Rule: every peripheral register pointer must be volatile

With volatile — CORRECT

```
// volatile = "this CAN change at any time
// without the compiler knowing why.
// Read it fresh every single time."
```

```
volatile uint32_t *SPI_SR =
    (volatile uint32_t*)0x40013008;

// Now the compiler MUST emit a real
// memory read instruction every loop.
while (*SPI_SR & (1<<1)) { }
// Correctly waits for hardware to clear
bit
```

```
// STM32 HAL does this for you:
// SPI->SR is already defined volatile
// in the CMSIS headers.
// This is one reason we use HAL
// instead of raw pointer arithmetic.
```

volatile also matters for variables shared between your main loop and an interrupt handler. Without it: stale cached value, silent bug.

Startup Sequence

The Vector Table

The first thing the CPU reads on every power-on

Vector table at 0x00000000

An array of 32-bit addresses in flash, right at the start

[0]	Initial SP	0x2001_FFFF	Top of SRAM → stack starts here
[1]	Reset	0x0800_0141	Reset handler address → CPU jumps here
[2]	NMI	0x0800_0179	Non-maskable interrupt
[3]	HardFault	0x0800_0195	Unrecoverable fault handler
[4]	...	...	More fault handlers
[16+]	SPI1_IRQ	0x0800_0265	SPI1 interrupt → radio receive
[16+]	TIM2_IRQ	0x0800_0281	Timer → election tick

main() can assume: SP is set, .data is initialised, .bss is zero. It immediately reads epoch_counter from EEPROM and begins HELLO_REQ broadcast.

Interrupts

Hardware telling your CPU 'stop what you're doing, handle this now'

Execution timeline

SPI byte received!

main loop: polling radio, checking timers, updating state...

resume

ISR: radio_recv_isr()

CPU saves registers, runs ISR, restores registers, continues exactly where it left off

HAVEN uses interrupts for:

SPI RX interrupt

Fills radio receive buffer without blocking election state machine

TIM2 interrupt

Fires every T_STABLE_MS — drives the stable_timer_callback()

EEPROM write complete

Signals when epoch_counter has been safely written to flash

Peripherals & Registers

How to Read a Peripheral Register

Worked example: SPI Status Register (SPI_SR)

SPI_SR at 0x40013008

(STM32 Reference Manual, Table 718)

31–9	8	7	6	5	4	3	2	1	0
Reserved	FRE Frame err	BSY Busy	OVR Overrun	MODF	CRCERR	UDR	CHSIDE	TXE TX empty	RXNE RX not empty

Raw register access (what HAL hides)

```
// Check if TX buffer is empty (bit 1)
volatile uint32_t *SPI_SR =
    (volatile uint32_t*)0x40013008;

if (*SPI_SR & (1u << 1)) {
    // TX buffer empty – safe to send
}

// Wait for RX data ready (bit 0)
while (!(*SPI_SR & (1u << 0))) {
    // spin until RXNE set by hardware
}

uint8_t received = *((volatile uint8_t*)
    0x4001300C); // SPI DR register
```

Same thing via STM32 HAL

```
// HAL defines all registers as structs:
// SPI1->SR is volatile uint32_t
// SPI1->DR is the data register

// Send one byte, 10ms timeout
HAL_StatusTypeDef result =
    HAL_SPI_Transmit(
        &hspi1,          // SPI handle
        &my_byte,        // data to send
        1,               // 1 byte
        10               // 10ms timeout
    );

if (result != HAL_OK) {
    // Error handling
}
```

HAVEN uses HAL_SPI_Transmit/Receive for the nRF24L01+ radio. You never write raw peripheral addresses — but you must understand what HAL is doing underneath.

SPI — How the Radio Chip is Connected

The nRF24L01+ talks to your STM32 over SPI

SPI timing — one byte transfer

4 wires: SCLK (clock), MOSI (master to slave), MISO (slave to master), CS (chip select, active low)

CS

SCLK

MOSI

DATA OUT (8 bits)

MISO

DATA IN (8 bits, offset by ~half clock)

nRF24L01+ pin connections

CE → GPIO PA8	Chip Enable — activate TX/RX
CSN → SPI1_NSS	Chip Select — starts SPI transaction
SCK → SPI1_SCK	Clock — STM32 drives this
MOSI → SPI1_MOSI	Data STM32 to radio
MISO → SPI1_MISO	Data radio to STM32
IRQ → GPIO PB0	Radio signals: data ready, TX done

Cross-Compilation & Toolchains

Cross-Compilation

Your laptop compiles code that runs on a different CPU architecture

main.c
hello.c
turbo.c

arm-none-eabi-gcc
-mcpu=cortex-m4
-mfpu=fpv4-sp-d16

.o object
files

arm-none-eabi-ld
+ linker script
main.ld

firmware.elf
(debug)
firmware.bin

Critical compiler flags — and what they do

-mcpu=cortex-m4 Target CPU — generates Thumb-2 instructions for that specific core

-mfloat-abi=hard Use FPU registers for float args (faster than soft-float emulation)

--specs=nosys.specs No OS system calls (no malloc from C library)

-mfpu=fpv4-sp-d16 Enable hardware floating-point unit (M4 only — omit for M0)

-Os Optimise for size — embedded flash is precious

Linker script (main.ld) — tells the linker where everything goes

```
MEMORY {  
    FLASH (rx) : ORIGIN=0x08000000,  
    LENGTH=512K  
    RAM (rwx) : ORIGIN=0x20000000,  
    LENGTH=128K  
}  
  
SECTIONS {
```

arm-none-eabi — what does the name mean?

arm Target architecture: ARM

none No OS (bare metal — 'none' vendor)

eabi Embedded Application Binary Interface

gcc/ld/objcopy Compiler / linker / binary converter

not gcc Your system gcc generates x86 — wrong CPU!

CMake for Embedded — the HAVEN Build System

How the project builds from source to .elf

CMakeLists.txt — key parts

```
cmake_minimum_required(VERSION 3.22)

# Tell CMake we're cross-compiling
set(CMAKE_SYSTEM_NAME Generic)
set(CMAKE_SYSTEM_PROCESSOR arm)
set(CMAKE_TRY_COMPILE_TARGET_TYPE
    STATIC_LIBRARY)

# Point to the cross-compiler
set(CMAKE_C_COMPILER arm-none-eabi-gcc)
set(CMAKE_OBJCOPY arm-none-eabi-objcopy)

project(haven C ASM)

# Source files
add_executable(firmware.elf
    src/main.c
    src/crypto/turbo_blom.c
    src/crypto/hmac_wrap.c
    src/protocol/hello.c
    src/protocol/election.c
    src/platform/radio_qemu.c
    startup/startup_stm32f4.c
```

Build commands

```
# Configure (once)
cmake -B build -DCMAKE_BUILD_TYPE=Release
.

# Build
cmake --build build

# Output
build/firmware.elf <- run in QEMU
build/firmware.bin <- flash to STM32
```

```
qemu-system-arm \
    -machine netduinoplus2 \
    -cpu cortex-m4 \
    -kernel build/firmware.elf \
    -serial tcp::4444,server \
    -nographic \
```

GDB attach

```
-S -s
# -serial -> TCP socket port 4444
```

```
arm-none-eabi-gdb build/firmware.elf
(gdb) target remote :1234
(gdb) break main
(gdb) continue
```

Debugging Without printf

Debugging Embedded Firmware

UART / Semihosting

UART: route `printf()` to a serial port.

On STM32: retarget `_write()` to `HAL_UART_Transmit`.

On QEMU: serial -> TCP socket -> netcat.

Semihosting: QEMU intercepts 'bkpt 0xAB' instructions and routes them to the host terminal. Zero hardware needed, but disabled on real hardware. Good for QEMU development.

```
// Retarget printf for UART:
int _write(int fd, char *buf, int len) {
    HAL_UART_Transmit(&huart2,
        (uint8_t*)buf, len, 100);
    return len;
}
```

GDB + QEMU / OpenOCD

GDB is a full debugger — breakpoints, watchpoints, step/next, inspect memory.

QEMU exposes GDB stub on port 1234:
-s flag in QEMU command

On real hardware: OpenOCD bridges SWD/JTAG debugger to GDB.

In GDB:

```
(gdb) break haven_hello.c:42
(gdb) watch neighbor_table[0].active
(gdb) x/8xw 0x20000000
(gdb) info registers
(gdb) backtrace
```

Watchpoints are powerful: "break when this variable changes to a bad value"

HardFault Handler

When your firmware crashes, the CPU jumps to `HardFault_Handler`.
By default: infinite loop. Useless.

Override it to dump useful info:

```
void HardFault_Handler(void) {
    uint32_t *sp =
        (uint32_t*)__get_MSP();
    uint32_t pc = sp[6]; // stacked PC
    uint32_t lr = sp[5]; // stacked LR

    printf("FAULT at PC=0x%08X", pc);
    while(1) {}
}

// PC tells you exactly where it crashed.
// Common causes: null deref, stack overflow, unaligned access, div/0
```

In QEMU: use GDB + semihosting. On physical hardware: use UART + the HardFault handler. Never rely on a blinking LED as your only debug output.

SECTION 07

QEMU + ns3 — Our Workflow

How QEMU and ns3 Work Together

One QEMU process per node — ns3 is the network between them

System architecture

QEMU Node 1

firmware.elf
election state
TCP:4441

QEMU Node 2

firmware.elf
election state
TCP:4442

QEMU Node 3

firmware.elf
TCP socket messages
election state
TCP:4443

ns3 process

graph topology
packet router
drop injector
metrics logger

Node 1 sends a CLAIM message

```
// firmware (Node 1)
radio_send(claim_msg, 46);
// -> writes 46 bytes to TCP:4441

// ns3 bridge
bytes = read_from(TCP:4441)
// Check: is Node 2 in radio range?
// Check: does drop_rate hit?
```

What ns3 controls

Topology

Which nodes are in range of each other (graph edges)

Drop rate

% of messages silently dropped on each link

Delay

Per-hop latency — affects convergence timing

Adversarial

Selectively drop CLAIM msgs from best candidate only

Metrics

Log every message: convergence rounds, msg count

mbedTLS — Crypto on Cortex-M

How HAVEN's HMAC operations actually run on firmware

Using HMAC-SHA256 in firmware

```
#include "mbedtls/md.h"

// Compute HMAC-SHA256
// key:      K_master or K_node
// msg:      data to authenticate
// out:      16-byte output (truncated to
// 128b)
int haven_hmac(
    const uint8_t *key, size_t key_len,
    const uint8_t *msg, size_t msg_len,
    uint8_t *out)
{
    mbedtls_md_context_t ctx;
    const mbedtls_md_info_t *info =
        mbedtls_md_info_from_type(
            MBEDTLS_MD_SHA256);

    mbedtls_md_init(&ctx);

    int ret = mbedtls_md_setup(
```

Stream A's first task: get this function returning correct output for known inputs.

```
    if (ret != 0) return ret;
```

How HAVEN uses this function

K_node derivation

hmac(K_master, node_id, 16) -> K_node
Done once at election start

Priority score

hmac(K_node, node_id XOR epoch, 16) -> priority
Verifiable by any neighbor

TurboBlom K_AB

XOR-based (no HMAC needed)
Pure binary field arithmetic

HELLO MAC

hmac(K_AB, HELLO_RESP fields, 16)
link_mac field in every authenticated msg

CLAIM link_mac

hmac(K_link[j], all preceding fields, 16)
Per-hop authentication

Timing on Cortex-M4 @ 168 MHz: ~10–30 us per HMAC

On M0 @ 48 MHz: ~80–150 us — still fast enough

Persisting the Epoch Counter

Why EEPROM matters and how to write to it safely

Why epoch_counter must survive power loss

If a node reboots and forgets its epoch counter, it restarts at 0. Every message it sends has epoch=0. Legitimate nodes see epoch mismatch and drop all messages — the rebooted node is permanently locked out.

Worse: an adversary who knows this can selectively power-cycle nodes.

Where it's stored

EEPROM (STM32L series)

Dedicated byte-addressable non-volatile memory. Ideal. Write single bytes, 100k+ cycles.

Emulated EEPROM (STM32F)

HAL_FLASHEx_DATAEEPROM_* functions. Uses flash pages with wear levelling.

FRAM (external)

Ferroelectric RAM — unlimited write cycles, fast. Optional external chip.

Safe write pattern

```
// NEVER write epoch_counter to flash
// on every message — flash endurance
// is 10,000-100,000 cycles.

// RULE: write only when epoch advances.

static uint32_t epoch_counter;

void epoch_advance(void) {
    epoch_counter++;

    // Write to emulated EEPROM
    HAL_FLASH_Unlock();
    HAL_FLASHEx_DATAEEPROM_Unlock();
    HAL_FLASHEx_DATAEEPROM_Program(
        FLASH_TYPEPROGRAMDATA_WORD,
        EPOCH_FLASH_ADDR,
        epoch_counter
    );
    HAL_FLASHEx_DATAEEPROM_Lock();
```

In QEMU: use a file on disk to simulate EEPROM — read on startup, write on epoch advance. Same abstraction, no hardware needed.

Putting It All Together

Every concept today owns a piece of HAVEN

MCU has no OS	→ Election state machine runs in main() polling loop. No threads. No scheduler.	All
Memory-mapped peripherals	→ SPI1 registers at 0x40013008 control the nRF24L01+ radio chip.	Stream B
volatile keyword	→ All peripheral register accesses in the SPI driver must be volatile.	Stream B
Vector table / interrupts	→ SPI_IRQHandler fills radio_buf. TIM2_IRQHandler calls stable_timer_callback().	Stream B
Flash RDP Level 2	→ K_master, node_id, key_rows in protected flash region. Unreadable post-deploy.	Stream A
Static allocation (no heap)	→ neighbor_table[8], dedup_cache[16], all fixed-size. Linker tells you if too big.	All
EEPROM write on event	→ epoch_counter written only when epoch advances. File-backed in QEMU.	Stream B
Cross-compilation	→ arm-none-eabi-gcc -mcpu=cortex-m4 builds firmware.elf from your C code.	All
mbedTLS on Cortex-M	→ haven_hmac() wraps mbedtls_md_hmac for K_node, K_AB, priority_mac, link_mac.	Stream A
radio_send / radio_recv	→ TCP socket in QEMU. SPI+nRF24 on physical hardware. Same election code.	Stream B
ns3 socket bridge	→ Routes packets between QEMU nodes. Injects drop, controls topology.	Stream C
GDB + QEMU + HardFault	→ Your primary debug tools for all firmware development.	All

Before You Write Code

Read these. Not skim. Read.

Memfault — Zero to Main (Parts 1-3)

interrupt.memfault.com/blog/zero-to-main-1

This is the long-form version of slides 9-10. It explains startup, linker scripts, and the vector table in exactly the right depth. Required before any firmware work.

QEMU ARM docs

qemu.org/docs/master/system/arm

How to launch your firmware image, expose UART as TCP, and attach GDB. Do this tutorial with an actual 'hello world' .elf before touching HAVEN code.

mbedTLS — MD module API

mbed-tls.readthedocs.io/en/latest/api/md/

Read the `mbedtls_md_hmac()` example. Implement it. Verify against NIST HMAC-SHA256 test vectors. If your output doesn't match, your crypto is wrong — stop and fix it.

nRF24L01+ datasheet

nordicsemi.com — product page, documents section

Read sections 6 (register map) and 7 (SPI timing). You need to know what the SPI commands mean before you write a driver or debug one.

STM32F4 Reference Manual (RM0090)

[st.com](https://www.st.com) -> STM32F4 -> Reference Manual

Find: SPI_SR register table, FLASH_CR register (for RDP), TIM2 control registers. Don't read all 1700 pages — use Ctrl+F like a datasheet.

First demo target (3-4 days): show something running in QEMU. One function, one output, verified correct. Not a design doc —